Freight Rate Prediction

Machine Learning Engineer technical assessment — modelling report

Development data: 48,000 labelled loads, Jan–Oct 2025
Final predictions: 12,000 loads, Nov–Dec 2025
Model: LightGBM, L1 objective on log rate · Code and reproduction steps in the repository README.

1. Executive summary

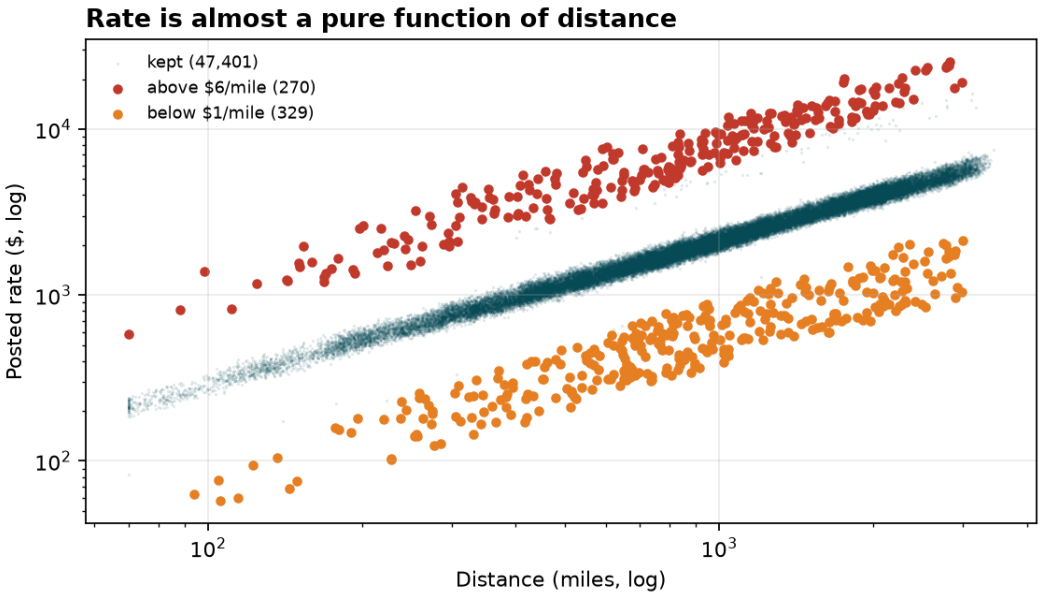
Posted rate is almost deterministic given distance: correlation of log(rate) with log(distance) is 0.99, and a gradient-boosted model on ten structural features reaches a holdout MAE of $52 and MAPE of 1.99% on unseen October loads. Two of the supplied columns — market_index and quote_signal — are actively harmful out of time: adding them to the model raises holdout MAE from $54 to $95. Both are level-shifted between the development window and the prediction window, so the model learns a relationship that does not transfer. I kept them out and documented the evidence rather than tuning around them.

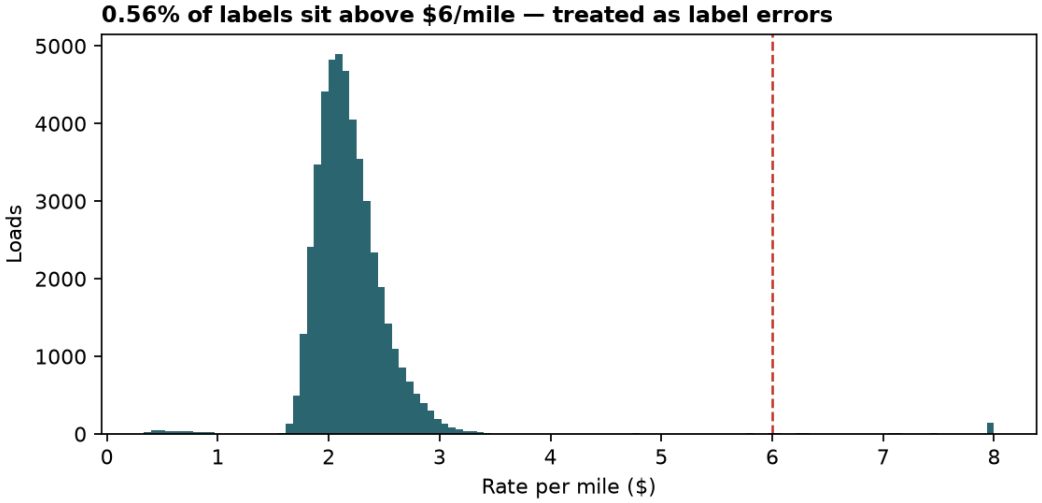
2. What the data looks like

One row per truckload: origin and destination (name plus coordinates), distance, equipment type, weight, date, two market-level signals, and the posted rate we have to predict. Development data covers ten months; the 12,000 validation loads cover November and December, so every validation fold is a genuine forward prediction rather than a random split.

2.1 Rate structure

- **Distance dominates:** correlation between log(rate) and log(distance) is 0.9919. Distance alone explains the shape of the problem.
- **Equipment is a level effect:** median $/mile is $2.05 for Dry Van, $2.22 for Flatbed, $2.31 for Reefer. Equipment shifts the level, not the slope.
- **Weekday premium:** Monday–Thursday average $2.16/mile, weekends $2.13. Real but small — worth 7% of holdout MAE.
- **Market index is a red herring:** market_index has a ten-month cycle (0.93 → 1.30 → 0.89) but correlates only 0.03 with rate at row level.





3. Data-quality issues and how I handled them

- **Impossible $/mile —** 599 loads (1.2%) price below $1.00 or above $6.00 per mile; the maximum is $14.13/mile on a 170-mile haul. These are label errors, not premium freight. Dropped from training, never from prediction. Kept the validation load anyway — a bad rate in the past says nothing about a new load's price.
- **Sign-flipped weight —** 292 rows carry negative weight (minimum −47,500 lb); 13 of them are exactly −47,500, a clean sign flip. Absolute value. The magnitude is right, the sign is a data-entry bug.
- **Missing values —** weight missing on 300 development rows (0.6%) and 165 validation rows; market_index missing on 374 development and 249 validation rows. Median imputation fitted

on development data only and reused unchanged for validation and December. With 0.6% missing, a model-based impute buys nothing.

- **Duplicate-looking loads —** 495 development rows share route, date and equipment — different weights and rates. Kept. Two shippers can move two loads the same lane the same day; deduplicating would delete real signal.
- **No coordinates in the December file —** december_chart_inputs.csv ships city names only, but the model uses coordinates. Built a 64-city lookup from the development data (each city maps to exactly one coordinate pair) and joined it back.

4. Validation design

The validation set is two future months, so a random K-fold split would leak information across time and produce a flattering score. I used rolling-origin validation instead: three expanding-window folds, each training on everything before a cut-off and predicting the single unseen month that follows it. The cut-offs are August, September and October 2025, which mirrors the one-to-two-month forward gap the real submission faces. Early stopping uses the fold's own holdout month, and I report the iteration count so the final fit on all ten months is not arbitrary.

Fold	Train rows	Best iter	MAE ($)	RMSE ($)	MAPE (%)	Bias ($)
train < 2025-08-01 -> 2025-08-01	33,303	486	53.56	204.14	2.23	22.97
train < 2025-09-01 -> 2025-09-01	38,004	553	56.38	214.87	2.18	-23.09
train < 2025-10-01 -> 2025-10-01	42,619	691	51.93	277.61	1.99	-27.55

Mean MAE across the three folds is $54 and mean MAPE 2.13%. The August fold is biased high and the later two biased low by a similar magnitude — that is market drift, not model error, and it is the reason I tested a level calibration step (section 6.3) before rejecting it.

5. Model and features

LightGBM with an L1 objective on log(rate). Trees capture the interactions (equipment × distance, lane × weekday) that a linear rate-per-mile model cannot, L1 loss matches the MAE metric being judged, and the log target keeps percentage error stable across a $57–$25,533 price range. Nine features, all available at quote time:

- **log(distance) —** the dominant driver, straight-line in logs
- **log(weight) —** capped below at 1,000 lb so the sign fix cannot produce −∞
- **equipment —** categorical — three levels, level shift on $/mile
- **pickup, delivery —** categorical lane effects: 64 cities, some lanes price structurally higher
- **lat/lon midpoint, north–south and east–west deltas —** coarse geography for lanes the model has never seen
- **day_of_week —** weekday premium, and the only feature that makes the December chart non-flat

Deliberately absent: market_index and quote_signal (section 6.2), and any date trend. A raw date feature would let the trees extrapolate a trend they never saw, which is exactly the failure mode that shows up in the last CV fold's RMSE.

6. What I tried and rejected

6.1 Feature ablation

Same three folds, same hyper-parameters, one feature group changed at a time:

Variant	MAE ($)	RMSE ($)	MAPE (%)	Bias ($)
A: core (distance, weight, equipment, pickup, delivery)	58.07	234.41	2.30	-9.48
B: A + geography	56.26	233.66	2.23	-10.16
C: B + day_of_week (chosen)	53.96	232.20	2.13	-9.22
D: C + market_index	95.48	259.00	3.81	-90.99
E: C + quote_signal	63.88	240.21	2.60	-5.66
F: C + market_index + quote_signal	84.62	251.98	3.44	-66.08

Geography is worth $1.81 of MAE, weekday $2.30. The chosen variant (C) is the best structural model I found.

6.2 Why market_index and quote_signal are excluded

In-window, market_index looks useful: its daily average correlates 0.87 with the daily model residual, and a row-level regression gives it a coefficient of 0.19. Out of window it collapses to 0.53 — and the damage is asymmetric. The development window cycles 0.81–1.40; the validation window sits at 0.92–0.94, near the bottom of that range but outside the range the model last saw. Trees then extrapolate a level the market is not paying. Ablation variant D makes this concrete: MAE goes from $53.96 to $95.48 and MAPE from 2.13% to 3.81% the moment market_index is added. quote_signal costs $9.92 of MAE for the same reason at smaller amplitude. Neither column survives contact with a forward split, so the honest move is to drop both and say why.

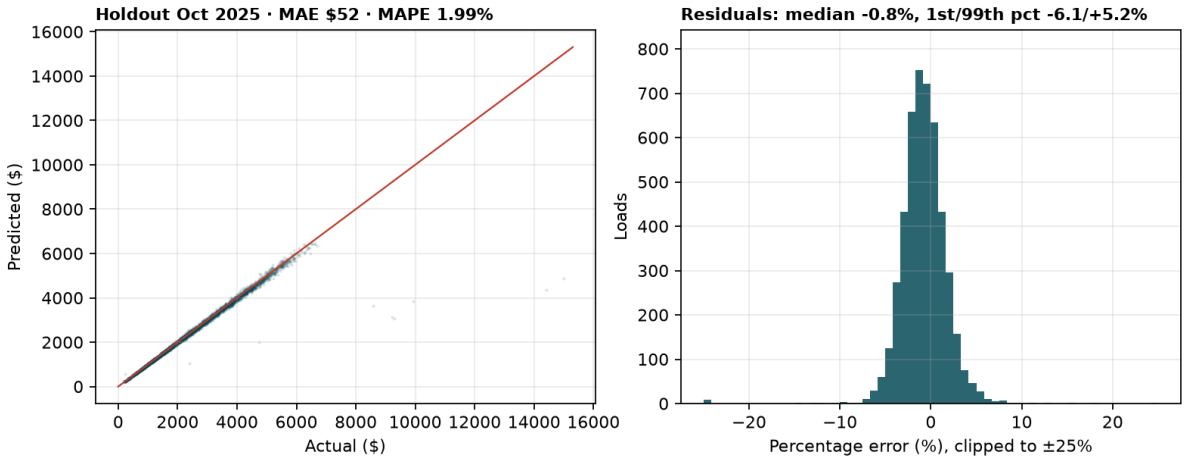
6.3 Level calibration — tested, rejected

Because the residual mean drifts between folds, I tested rescaling predictions by the mean residual of the last 30/45/60/90 days of training data. It helps one fold, hurts another, and on average makes MAE worse ($56.71 → $74.80 across folds at 60 days). The drift is not predictable from the development window, so an offset is a coin flip with extra steps. Predictions ship uncalibrated.

6.4 Other variants

- **Objective:** L1 on log target won ($53.72 vs $54.63 for L2, $54.24 for Huber). Small but consistent across folds.
- **Seasonality:** seasonal index on the daily residual, 1–3 harmonics. Extrapolating into November–December was worse than flat in three of four backtests; the December chart is therefore driven by weekday only.
- **Rate-per-mile baseline:** plain median $/mile by equipment scored 8.19% MAPE against the model's 2.13% — a useful floor, not a competitor.

7. Results

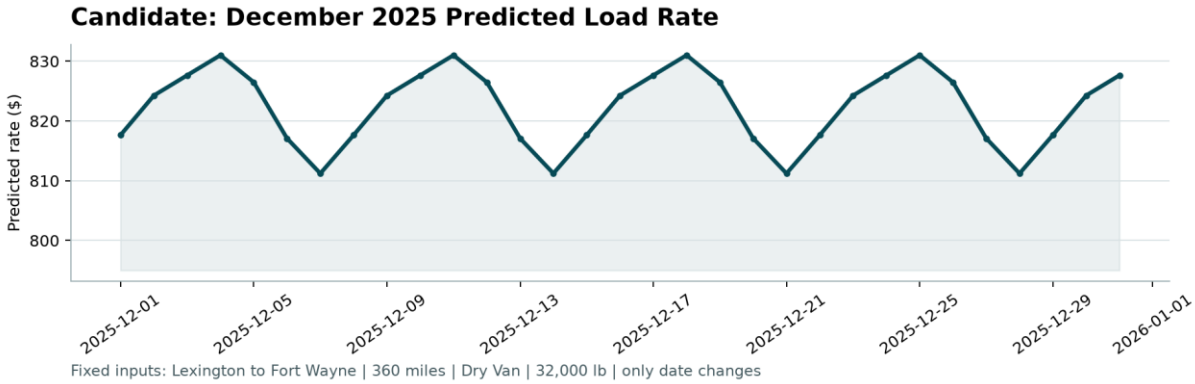


Final fold (train through September, predict October): MAE $52, RMSE $278, MAPE 1.99%, bias $-28 on 4,782 loads. Residuals are close to symmetric with a small number of high-rate lanes in the tail; that tail is what separates MAE from RMSE by a factor of five.

Distance band (miles)	Loads	MAPE (%)
(0, 500]	1,060	2.41
(500, 1000]	1,498	1.73
(1000, 1500]	877	1.87
(1500, 3500]	1,347	2.03

Short hauls are the hardest: under 500 miles, MAE is dominated by fixed accessorials that the feature set does not observe (loading time, detention, fuel surcharge floors). If this model went to production, that band is where I would add data, not where I would add model capacity.

8. December chart



Produced by running score.py on outputs/december_chart_inputs.csv. Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb, only the date changes: $811 to $831, mean $822. The shape is a weekly cycle peaking Wednesday and bottoming Sunday, which is the weekday effect the model learned from ten months of data. It is deterministic because every feature except day_of_week is held

constant by construction — the model has no residual noise to reproduce. For reference, the 32 historical Lexington to Fort Wayne loads average $857 because their weights run 16,000–41,000 lb; the fixed 32,000 lb chart point sits in that range at the weekday-adjusted $/mile for that lane.

9. Limitations and what I would do next

- **No market-level regressor:** the validation months sit at a different market level than the training window. Any model that ignores the level is unbiased on average but wrong on any given week; recovering that level needs external data (DAT/FreightWaves indices), not more trees.
- **No transit or lead time:** a load quoted on 30 December but picked up in January is priced off January conditions. The model uses quote date only.
- **No accessorial features:** fixed accessorials are unobserved, and they are the reason short hauls carry the worst percentage error.
- **Hyper-parameters:** one LightGBM configuration with early stopping, not a hyper-parameter search. The CV spread across folds ($52–$56 MAE) is wider than the gap between reasonable configurations, so a search would tune to fold noise.
- **Next step:** probabilistic output for a quote tool. Quantile objectives at 10/50/90 would give a price band instead of a point, which is what a pricing team actually needs.

10. Reproducing this

Repository layout and the three commands that regenerate every artefact:

- `data/` `train_test.csv, validation.csv, december_chart_inputs.csv, validation_predictions_template.csv`
- `pipeline.py` `clean → features → fit → outputs/validation_predictions.csv + december_chart_inputs.csv`
- `ablation.py` `feature-set ablation (section 6.1)`
- `make_report_assets.py` `figures and statistics for this report`

```
python -m pip install -r requirements.txt
python pipeline.py --cv
python score.py --predictions outputs/validation_predictions.csv --december-predictions
outputs/december_chart_inputs.csv
```

The scorer validates row count, ID coverage and positivity, then writes scorer_results/candidate_december.png.